

MongoDB — SOC-Lite

Séance 2 — Opérations CRUD avancées avec MongoDB

Partie	Objectif	Statut
TP1	Démarrage Docker & connexion MongoDB	✓ Complété
TP2	Insertion de données (insertMany)	✓ Complété
TP3	Lecture et vérification des données	✓ Complété
TP4	Mise à jour d'incidents et commentaires	✓ Complété
TP5	Création d'incident lié — charlie	✓ Complété
TP6	Requêtes avancées et filtres complexes	✓ Complété

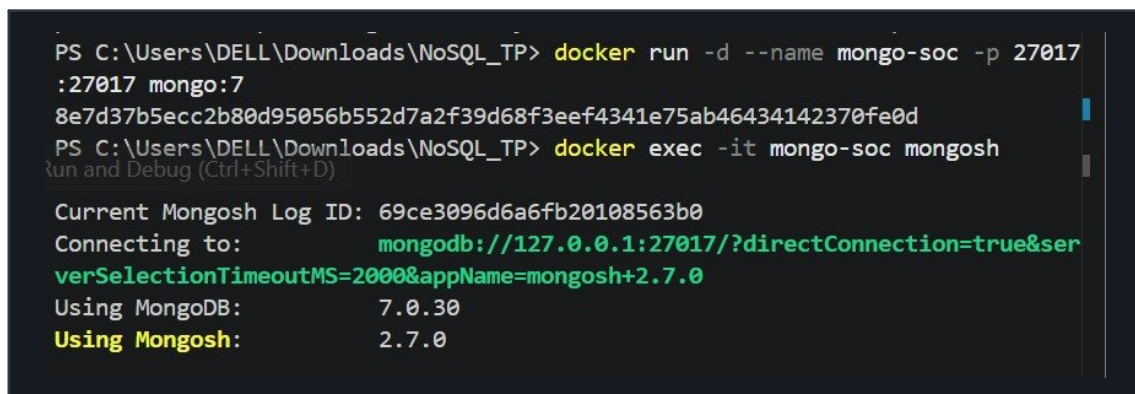
TP1 — Démarrage de l'environnement Docker

Cette première partie couvre le lancement du conteneur MongoDB via Docker et l'établissement de la connexion à l'aide de **mongosh**. Le conteneur est nommé **mongo-soc** et expose le port **27017**.

Commandes exécutées :

```
docker run -d --name mongo-soc -p 27017:27017 mongo:7  
docker exec -it mongo-soc mongosh
```

Résultat obtenu :



```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker run -d --name mongo-soc -p 27017  
:27017 mongo:7  
8e7d37b5ecc2b80d95056b552d7a2f39d68f3eef4341e75ab46434142370fe0d  
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo-soc mongosh  
Run and Debug (Ctrl+Shift+D)  
  
Current Mongosh Log ID: 69ce3096d6a6fb20108563b0  
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&ser  
verSelectionTimeoutMS=2000&appName=mongosh+2.7.0  
Using MongoDB:      7.0.30  
Using Mongosh:       2.7.0
```

■ Lancement du conteneur Docker et connexion via mongosh — MongoDB 7.0.30 / Mongosh 2.7.0

La connexion s'effectue vers <mongodb://127.0.0.1:27017> en mode `directConnection`. La version MongoDB est **7.0.30** et Mongosh **2.7.0**.

TP2 — Insertion des données initiales

Cette partie consiste à peupler la base **soc_lite** avec trois collections : **users** (4 documents), **security_logs** (8 documents) et **incidents** (2 documents). L'opération **insertMany** est utilisée pour chaque collection.

2.1 — Insertion des utilisateurs (users)

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69ce30f9d6a6fb20108563b1'),
    '1': ObjectId('69ce30f9d6a6fb20108563b2'),
    '2': ObjectId('69ce30f9d6a6fb20108563b3'),
    '3': ObjectId('69ce30f9d6a6fb20108563b4')
  }
}
soc_lite> 
```

■ insertMany — 4 utilisateurs insérés dans db.users

2.2 — Insertion des journaux de sécurité (security_logs)

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69ce31b3d6a6fb20108563b5'),
    '1': ObjectId('69ce31b3d6a6fb20108563b6'),
    '2': ObjectId('69ce31b3d6a6fb20108563b7'),
    '3': ObjectId('69ce31b3d6a6fb20108563b8'),
    '4': ObjectId('69ce31b3d6a6fb20108563b9'),
    '5': ObjectId('69ce31b3d6a6fb20108563ba'),
    '6': ObjectId('69ce31b3d6a6fb20108563bb'),
    '7': ObjectId('69ce31b3d6a6fb20108563bc')
  }
}
soc_lite> 
```

■ insertMany — 8 logs de sécurité insérés dans db.security_logs

2.3 — Insertion des incidents

```
|      '','  
|      mitre_tactics: ["TA0010", "TA0011"],  
|      mitre_techniques: ["T1041", "T1071.001"]  
|    }  
|  ])  
|}  
|  acknowledged: true,  
|  insertedIds: {  
|    '0': ObjectId('69ce32b0d6a6fb20108563be'),  
|    '1': ObjectId('69ce32b0d6a6fb20108563bf')  
|  }  
|}  
soc_lite> |
```

■ *insertMany* — 2 incidents insérés dans db.incidents

Chaque insertion retourne un objet avec `acknowledged: true` et les `ObjectId` générés automatiquement par MongoDB pour chaque document.

TP3 — Vérification et lecture des données

Une fois les données insérées, cette partie vérifie le nombre de documents dans chaque collection via `countDocuments()` et consulte le détail d'un incident spécifique avec `findOne()`.

Commandes de vérification :

```
db.users.countDocuments() → 4
db.security_logs.countDocuments() → 8
db.incidents.countDocuments() → 2
```

Détail de l'incident INC-2025-001 :

```
soc_lite> db.users.countDocuments()
4
soc_lite> db.security_logs.countDocuments()
8
soc_lite> db.incidents.countDocuments()
2
soc_lite> db.incidents.findOne({ incident_id: "INC-2025-001" })
{
  _id: ObjectId('69ce32b0d6a6fb20108563be'),
  incident_id: 'INC-2025-001',
  title: 'Brute force SSH sur srv-web-01',
  description: 'Multiples tentatives SSH échouées depuis 192.168.1.105 ciblant root et admin.',
  severity: 'high',
  status: 'investigating',
  created_at: ISODate('2025-03-03T10:30:00.000Z'),
  updated_at: ISODate('2025-03-03T11:00:00.000Z'),
  assigned_to: 'alice_analyst',
  tags: [ 'brute-force', 'ssh', 'external' ],
  related_log_ids: [
    ObjectId('69ce31b3d6a6fb20108563b5'),
    ObjectId('69ce31b3d6a6fb20108563b6'),
    ObjectId('69ce31b3d6a6fb20108563b7')
  ],
  comments: [
    {
      author: 'alice_analyst',
      text: '3 tentatives en 2s depuis la même IP. Pattern brute-force confi'
```

■ `countDocuments()` + `findOne({ incident_id: 'INC-2025-001' })` — Brute force SSH sur srv-web-01

L'incident **INC-2025-001** correspond à une attaque brute force SSH depuis **192.168.1.105** ciblant les comptes **root** et **admin**. Sa sévérité est **high** et son statut *investigating*.

TP4 — Mise à jour des incidents et commentaires

Cette partie illustre les opérations de mise à jour : ajout de commentaires à un incident existant via `$push` et insertion de nouveaux logs avec des IDs de référence. La consultation des commentaires enrichis est ensuite effectuée.

4.1 — Ajout de commentaires et vérification

```

    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
soc_lite> db.incidents.findOne(
  |   { incident_id: "INC-2025-001" },
  |   { comments: 1, incident_id: 1 }
  | )
{
  _id: ObjectId('69ce32b0d6a6fb20108563be'),
  incident_id: 'INC-2025-001',
  comments: [
    {
      author: 'alice_analyst',
      text: '3 tentatives en 2s depuis la même IP. Pattern brute-force confi
rmé.',
      timestamp: ISODate('2025-03-03T10:35:00.000Z')
    },
    {
      author: 'bob_senior',
      text: 'IP bloquée au firewall. Surveiller si rebond depuis un autre ve
cteur.',
      timestamp: ISODate('2025-03-03T10:50:00.000Z')
    },
    {
      author: 'diana_junior',
      text: "L'IP 192.168.1.105 appartient au sous-réseau visiteur. Aucun co
mpte légitime.",
      timestamp: ISODate('2026-04-02T09:15:13.135Z')
    }
  ]
}

```

■ `updateOne` avec `$push` — Ajout commentaires à INC-2025-001 + projection des comments

Trois analystes ont commenté l'incident : **alice_analyst** (pattern brute-force confirmé), **bob_senior** (IP bloquée au firewall), et **diana_junior** (IP appartenant au sous-réseau visiteur).

4.2 — Insertion de logs supplémentaires

```

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69ce33edd6a6fb20108563c0'),
    '1': ObjectId('69ce33edd6a6fb20108563c1'),
    '2': ObjectId('69ce33edd6a6fb20108563c2')
  }
}
soc_lite> 

```

■ `insertMany` — 3 nouveaux documents insérés dans `security_logs`

TP5 — Création d'un incident lié : escalade charlie

Cette partie crée un incident critique (INC-2025-003) lié à une tentative d'escalade de privilèges par l'utilisateur **charlie** depuis l'IP **10.0.1.200**. Les logs correspondants sont récupérés dynamiquement et référencés dans l'incident.

5.1 — Création de l'incident INC-2025-003

```
soc_lite> let charlieLogIds = db.security_logs.find({
|   src_ip: "10.0.1.200",
|   timestamp: { $gte: new Date("2025-03-03T17:00:00Z") }
| }).toArray().map(d => d._id)

| db.incidents.insertOne({
|   incident_id: "INC-2025-003",
|   title: "Tentative d'escalade de privilèges - charlie",
|   description: "Échecs de connexion suivis d'une tentative d'escalade depuis 10.0.1.200.",
|   severity: "critical",
|   status: "new",
|   created_at: new Date(),
|   updated_at: new Date(),
|   assigned_to: "alice_analyst",
|   tags: ["privilege-escalation", "internal", "account-lockout"],
|   related_log_ids: charlieLogIds,
|   comments: [],
|   mitre_tactics: ["TA0004"],
|   mitre_techniques: ["T1078"]
| })
{
  acknowledged: true,
  insertedId: ObjectId('69ce3485d6a6fb20108563c5')
}
soc_lite> 
```

■ Récupération des charlieLogIds + insertOne INC-2025-003 — MITRE TA0004/T1078

L'incident est tagué **critical** avec les tactiques MITRE **TA0004** (Privilege Escalation) et la technique **T1078** (Valid Accounts). Il est assigné à **alice_analyst**.

5.2 — Vérification des logs liés

```
soc_lite> let inc3 = db.incidents.findOne({ incident_id: "INC-2025-003" })
| db.security_logs.find({ _id: { $in: inc3.related_log_ids } }).count()
3
soc_lite> 
```

■ Vérification : 3 logs liés à INC-2025-003 via related_log_ids

5.3 — Assignment de plusieurs analystes


```
soc_lite> db.incidents.updateOne(
|   { incident_id: "INC-2025-002" },
|   { $addToSet: { assigned_analysts: { $each: ["bob_senior","alice_analyst"
|   ] } } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
soc_lite> █
```

■ *updateOne avec \$addToSet — bob_senior et alice_analyst assignés à INC-2025-002*

TP6 — Requêtes avancées et filtres complexes

La dernière partie explore des requêtes de lecture plus sophistiquées : filtrage par sévérité, plages temporelles avec `$gte/$lte`, projections de champs, et tri/limitation des résultats.

6.1 — Logs de sévérité critique

```
soc_lite> db.security_logs.find({ severity: "critical" })
[
  {
    _id: ObjectId('69ce31b3d6a6fb20108563ba'),
    timestamp: ISODate('2025-03-03T14:22:10.000Z'),
    event_type: 'malware_detected',
    src_ip: '192.168.2.30',
    dst_ip: '10.0.0.10',
    src_port: 80,
    dst_port: 8080,
    protocol: 'HTTP',
    user_attempted: null,
    device_id: 'endpoint-pc-042',
    severity: 'critical',
    raw_log: 'Trojan.GenericKD.46543210 detected in /tmp/update.exe'
  },
  {
    _id: ObjectId('69ce31b3d6a6fb20108563bc'),
    timestamp: ISODate('2025-03-03T16:45:00.000Z'),
    event_type: 'data_exfiltration_suspect',
    src_ip: '10.0.0.10',
    dst_ip: '203.0.113.50',
    src_port: 443,
    dst_port: 443,
    protocol: 'HTTPS',
    user_attempted: null,
    device_id: 'firewall-main',
    severity: 'critical',
    raw_log: 'Unusual outbound transfer 2.3GB to 203.0.113.50'
  },
  {

```

■ `find({ severity: 'critical' })` — `malware_detected` + `data_exfiltration_suspect`

Deux événements critiques ressortent : un **Trojan.GenericKD** détecté dans `/tmp/update.exe` sur `endpoint-pc-042`, et une exfiltration suspecte de **2.3 GB** vers l'IP externe **203.0.113.50**.

6.2 — Logs sur plage horaire (10h–12h)

```
soc_lite> db.security_logs.find(
|   timestamp: { $gte: new Date("2025-03-03T10:00:00Z"),
|               $lte: new Date("2025-03-03T12:00:00Z") }
| }).sort({ timestamp: 1 })
[
  {
    _id: ObjectId('69ce31b3d6a6fb20108563b5'),
    timestamp: ISODate('2025-03-03T10:05:22.000Z'),
    event_type: 'login_failed',
    src_ip: '192.168.1.105',
    dst_ip: '10.0.0.2',
    src_port: 44821,
    dst_port: 22,
    protocol: 'SSH',
    user_attempted: 'root',
    device_id: 'srv-web-01',
    severity: 'medium',
    raw_log: 'Failed password for root from 192.168.1.105 port 44821 ssh2'
  },
  {
    _id: ObjectId('69ce31b3d6a6fb20108563b6'),
    timestamp: ISODate('2025-03-03T10:05:23.000Z'),
    event_type: 'login_failed',
    src_ip: '192.168.1.105',
    dst_ip: '10.0.0.2',
    src_port: 44822,
    dst_port: 22,
    protocol: 'SSH',
    user_attempted: 'root',
    device_id: 'srv-web-01',
    severity: 'medium',
    raw_log: 'Failed password for root from 192.168.1.105 port 44822 ssh2'
  }
]
```

■ `find()` avec `$gte/$lte` sur `timestamp` — triés par `timestamp` ASC

6.3 — Projection sur les logins échoués

```
soc_lite> db.security_logs.find(
|   { event_type: "login_failed" },
|   { src_ip: 1, user_attempted: 1, timestamp: 1, _id: 0 }
| )
[
  {
    timestamp: ISODate('2025-03-03T10:05:22.000Z'),
    src_ip: '192.168.1.105',
    user_attempted: 'root'
  },
  {
    timestamp: ISODate('2025-03-03T10:05:23.000Z'),
    src_ip: '192.168.1.105',
    user_attempted: 'root'
  },
  {
    timestamp: ISODate('2025-03-03T10:05:24.000Z'),
    src_ip: '192.168.1.105',
    user_attempted: 'admin'
  },
  {
    timestamp: ISODate('2025-03-03T15:00:00.000Z'),
    src_ip: '10.0.1.200',
    user_attempted: 'charlie'
  },
  {
    timestamp: ISODate('2025-03-03T17:00:00.000Z'),
    src_ip: '10.0.1.200',
    user_attempted: 'charlie'
  }
]
```

■ `find({ event_type: 'login_failed' })` avec projection `src_ip`, `user_attempted`, `timestamp`

La projection permet de n'afficher que les champs utiles : IP source, utilisateur tenté et horodatage. On identifie clairement les attaques de **192.168.1.105** sur `root/admin` et les tentatives de **charlie** depuis **10.0.1.200**.

6.4 — Derniers événements (tri DESC + limite)

```
soc_lite> db.security_logs.find().sort({ timestamp: -1 }).limit(3)
[
  {
    _id: ObjectId('69ce33edd6a6fb20108563c2'),
    timestamp: ISODate('2025-03-03T17:01:00.000Z'),
    event_type: 'privilege_escalation',
    src_ip: '10.0.1.200',
    dst_ip: '10.0.0.5',
    src_port: 61310,
    dst_port: 443,
    protocol: 'HTTPS',
    user_attempted: 'charlie',
    device_id: 'siem-console',
    severity: 'critical',
    raw_log: 'Privilege escalation attempt detected for charlie'
  },
  {
    _id: ObjectId('69ce33edd6a6fb20108563c1'),
    timestamp: ISODate('2025-03-03T17:00:05.000Z'),
    event_type: 'login_failed',
    src_ip: '10.0.1.200',
    dst_ip: '10.0.0.5',
    src_port: 61305,
    dst_port: 443,
    protocol: 'HTTPS',
    user_attempted: 'charlie',
    device_id: 'siem-console',
    severity: 'medium',
    raw_log: 'Failed login for charlie - account locked'
  },
  {

```

■ `find().sort({ timestamp: -1 }).limit(3)` — 3 événements les plus récents

Les 3 événements les plus récents montrent la séquence d'attaque de charlie : **privilege_escalation** (critique) puis **login_failed** (compte verrouillé) sur le *siem-console* via HTTPS depuis le port 61310/61305.

Conclusion

Partie	Compétences acquises
TP1	Déploiement de MongoDB via Docker, connexion mongosh
TP2	Insertion en masse (insertMany) sur plusieurs collections
TP3	Comptage et lecture ciblée de documents (countDocuments, findOne)
TP4	Mise à jour documentaire avec \$push, projection de champs
TP5	Références dynamiques entre collections, \$addToSet, MITRE ATT&CK
TP6	Filtres avancés (\$gte/\$lte), projection, sort, limit

Ces travaux pratiques ont permis de maîtriser les opérations CRUD fondamentales et avancées de MongoDB dans un contexte réaliste de gestion de logs de sécurité (SOC). La modélisation flexible des documents JSON et la puissance des opérateurs MongoDB (`$push`, `$addToSet`, `$gte`, `$lte`, `$in`) ont été mis en pratique sur un jeu de données simulant un vrai environnement de cybersécurité.